

DevOps Interview Q&A Answers

3–5 Years Experience Level • 17 Questions Covered

AWS • EKS • Terraform • Jenkins • Docker • Kubernetes • Monitoring • Incident Response

Q1 Tell me about yourself.

BACKGROUND

B.Tech in Aeronautical Engineering (2025, RTM Nagpur University) with ~9 months of hands-on DevOps internship at Hisan Labs.

CORE SKILLS

- Cloud: AWS (EKS, EC2, VPC, S3, IAM, RDS, ECR, CloudFront, Route53, Lambda)
- IaC & CI/CD: Terraform, Jenkins, GitHub Actions
- Containers: Docker, Kubernetes, Helm, Kustomize
- Monitoring: Datadog, Prometheus, Grafana, ELK Stack
- Security: SonarQube, Trivy; Scripting: Python, Bash

FLAGSHIP PROJECT

ApnaKart — 10-microservice cloud-native e-commerce platform on AWS EKS. Key outcomes:

- 77% reduction in deployment time via Jenkins pipelines
- 99.9% uptime maintained across environments
- Zero critical CVEs through Trivy + SonarQube gates
- 30% MTTR reduction using Datadog APM and alerting

Tip: Close with your goal — e.g., 'I'm actively targeting DevOps/SRE roles and ready to own production-grade infra from day one.'

Q2 Explain your project architecture.

APNAKART — ARCHITECTURE OVERVIEW

- **Frontend:** React 18 app served via CloudFront + S3 (static hosting)
- **Backend:** 10 Spring Boot microservices — each with its own MySQL 8.0 RDS instance (database-per-service pattern)
- **Compute:** AWS EKS cluster provisioned via Terraform; services deployed as Kubernetes Deployments with HPA
- **Networking:** Custom VPC with public/private subnets; Route53 for DNS; ALB Ingress Controller for traffic routing
- **Image Registry:** DockerHub for dev/test; ECR for UAT/prod (dual registry strategy)
- **CI/CD:** Jenkins with branch-to-environment strategy — dev → test → uat → prod promotion gates
- **Config Management:** Kustomize overlays per environment; Helm for third-party charts
- **Security:** IRSA/OIDC for pod-level AWS permissions; Trivy image scans; SonarQube quality gates
- **Observability:** Datadog DaemonSet APM + dashboards; Prometheus + Grafana for cluster metrics; ELK for log aggregation

KEY DESIGN DECISIONS

- Database-per-service for isolation and independent scaling
- IRSA eliminates static AWS credentials in pods
- Kustomize overlays keep base manifests DRY across environments

Q3 Why do you use EKS instead of ECS?

EKS ADVANTAGES FOR APNAKART

- **Industry standard:** Kubernetes is the dominant orchestration platform; skills are portable across clouds
- **Ecosystem richness:** Helm, Kustomize, Argo CD, KEDA, and the broader CNCF toolchain integrate natively
- **Fine-grained control:** Custom Resource Definitions (CRDs), admission webhooks, network policies — ECS abstracts these away
- **Multi-cloud portability:** The same manifests can target GKE or AKS with minimal changes; ECS is AWS-only
- **Advanced scheduling:** Node affinity, taints/tolerations, and pod disruption budgets give precise placement control

WHEN ECS IS THE RIGHT CHOICE

- Simpler workloads with no Kubernetes expertise on the team
- Faster time-to-production with Fargate (no node management)
- Lower operational overhead for small teams

Honest answer: ECS with Fargate has a lower ops burden. EKS was chosen because ApnaKart needed Kustomize-based multi-env management and the Datadog/Prometheus integration ecosystem.

Q4 Have you handled cloud cost or budget management?

COST CONTROLS IMPLEMENTED

- **AWS Cost Explorer:** Weekly cost reviews per service tag; identified idle RDS instances and right-sized them
- **Resource tagging strategy:** Every resource tagged with env, team, project — enables per-team cost attribution
- **Auto-scaling:** HPA on EKS workloads to scale down during off-hours; Cluster Autoscaler to remove idle nodes
- **Spot/On-demand mix:** Non-critical workloads (dev/test) run on Spot instances — ~60% EC2 cost reduction in lower envs
- **S3 lifecycle policies:** Old artifacts and logs moved to S3-IA or Glacier after defined retention periods
- **Budget alerts:** AWS Budgets configured with SNS notifications at 80% and 100% of monthly threshold

TERRAFORM COST HYGIENE

- terraform destroy scheduled for non-production environments outside working hours
- Instance type decisions tracked in PR comments before apply

Q5 Explain how you set up monitoring tools (CloudWatch, Grafana, etc.).

DATADOG (PRIMARY APM)

- Deployed Datadog Agent as a DaemonSet on EKS nodes via Helm chart
- Configured APM tracing in Spring Boot services using dd-java-agent
- Created dashboards: p99 latency, error rate, throughput per microservice
- Alerts: PagerDuty integration for P1/P2 thresholds

PROMETHEUS + GRAFANA

- kube-prometheus-stack deployed via Helm; scrapes node-exporter, kube-state-metrics
- Custom ServiceMonitor CRDs added for Spring Boot Actuator /metrics endpoints
- Grafana dashboards: cluster CPU/memory, pod restarts, JVM heap, GC pause time

ELK STACK

- Filebeat DaemonSet ships container logs to Elasticsearch
- Logstash pipelines parse structured JSON logs; Kibana for search and dashboards
- Index lifecycle management (ILM) to rotate and archive old indices

CLOUDWATCH

- RDS Enhanced Monitoring, VPC Flow Logs, and EKS control-plane logs sent to CloudWatch
- CloudWatch Alarms on RDS CPU > 80%, freeable memory < 20%

Q6 Write a Bash script to check disk usage and CPU usage.

Q8

What would you do if the Terraform state file is corrupted?

IMMEDIATE STEPS

- **Step 1 — Do not run terraform apply.** A corrupted state + apply can duplicate or destroy resources
- **Step 2 — Check remote backend versioning.** S3 backend with versioning enabled? Retrieve the last known-good version
- **Step 3 — Restore the backup:**
 - `aws s3api list-object-versions --bucket tf-state-bucket --prefix terraform.tfstate`
 - `aws s3api get-object --bucket tf-state-bucket --key terraform.tfstate --version-id <VERSION> terraform.tfstate`
- **Step 4 — terraform state list** to verify resources are still tracked correctly
- **Step 5 — If no backup exists:** Use terraform import to re-import existing resources one by one

PREVENTION (should say proactively)

- S3 backend with versioning + DynamoDB state locking enabled by default in all our workspaces
- State file never stored locally or in Git
- terraform state pull regularly backed up before risky operations

Q9

Infrastructure created outside Terraform — how do you handle it?

DETECTION

- AWS Config Rules or CloudTrail alerts notify when resources are created without Terraform tags
- Regular terraform plan in CI pipeline surfaces unexpected drift as plan differences

REMEDATION OPTIONS

- **Option A — Import into state (preferred):**
 - `terraform import aws_instance.rogue i-0abc123def456`
 - Then write the matching HCL resource block and run terraform plan to confirm zero diff
- **Option B — Destroy the rogue resource** if it violates policy and has no production impact
- **Option C — Exclude via ignore_changes** if the resource must exist but Terraform shouldn't manage it

PROCESS IMPROVEMENT

- Enforce IAM SCPs or permission boundaries so engineers cannot create resources outside IaC pipelines
- Add drift detection to CI: scheduled terraform plan with SNS alert on non-empty plan

Q10

What steps have you taken to prevent infrastructure from being deleted unintentionally?

TERRAFORM SAFEGUARDS

- **lifecycle { prevent_destroy = true }** on critical resources — EKS cluster, RDS, S3 buckets
- **S3 versioning + MFA Delete** enabled on state bucket and data buckets
- **terraform plan output review** mandatory in PR before any apply in prod; destruction requires separate approval step

AWS-LEVEL PROTECTION

- **Termination protection** enabled on EC2 instances and RDS clusters via AWS console + enforced in Terraform
- **S3 Object Lock** on critical buckets (compliance mode for audit logs)
- **IAM permission boundaries** prevent engineers from running destroy in prod accounts directly
- **Resource-based policies** + SCPs (Service Control Policies) in AWS Organizations to restrict destructive actions

PROCESS CONTROLS

- Prod Terraform workspace requires two-person approval via pull request review
- Automated backup and snapshot policies for RDS (7-day retention) and EBS volumes

Q11

What do you do if a Jenkins pipeline fails?

IMMEDIATE TRIAGE

- **Step 1 — Identify the failing stage:** Check Jenkins console output; look for the first red stage in the pipeline view
- **Step 2 — Classify the failure type:**
 - Build failure → compilation error, dependency issue
 - Test failure → unit/integration test regression
 - Docker build failure → Dockerfile issue, base image pull rate limit
 - Deploy failure → Kubernetes manifest error, image not found, quota exceeded
 - Infrastructure failure → AWS API throttling, credentials expired

RESOLUTION STEPS

- Reproduce locally first — run the same command that failed in the pipeline
- For flaky tests: re-run the stage; if intermittent, add retry logic in Jenkinsfile
- For credential issues: check Jenkins credential store and IAM role attached to Jenkins node
- For Kubernetes deploy failure: `kubectl describe pod` and `kubectl logs` for the failing pod

POST-FIX

- Add a Slack/email notification step to Jenkinsfile for immediate alert on failure
- Document root cause in the PR or Jira ticket; update runbook if it's a recurring pattern

Q12 What is the difference between CMD and RUN in Docker?

RUN

- Executes a command **during image build** and commits the result as a new layer
- Used for installing dependencies, configuring the OS, creating directories
- Each RUN creates a new layer — combine commands with && to reduce layer count

CMD

- Defines the **default command to run when a container starts** — not executed at build time
- Can be overridden at runtime: `docker run myimage /bin/bash` overrides CMD
- Only the last CMD in a Dockerfile takes effect

KEY DIFFERENCE TABLE

Aspect	RUN	CMD
When it runs	Image build time	Container start time
Purpose	Install / configure	Default start command
Layers created	Yes — each RUN adds a layer	No new layer
Override at runtime	No	Yes — <code>docker run <cmd></code>

ENTRYPOINT vs CMD: ENTRYPOINT sets the executable that cannot be overridden easily; CMD provides default args to ENTRYPOINT.

Q13 Explain how you set up an automatic trigger in Jenkins when code is pushed to GitHub.

METHOD 1 — GITHUB WEBHOOK (RECOMMENDED)

- **Jenkins side:** In the job/pipeline config, enable 'GitHub hook trigger for GITScm polling'
- **GitHub side:** Settings → Webhooks → Add webhook
 - Payload URL: `http://<jenkins-url>/github-webhook/`
 - Content type: `application/json`
 - Events: Just the push event (or `pull_request` for PR builds)
- Jenkins receives the push event and triggers the pipeline automatically

METHOD 2 — JENKINSFILE DECLARATIVE TRIGGER


```

pipeline {
  agent any
  triggers {
    githubPush() // fires on every push
    // pollSCM('H/5 * * * *') // fallback: poll every 5 mins
  }
  stages {
    stage('Build') { steps { sh 'mvn clean package' } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Docker Build') {
      steps {
        sh 'docker build -t $ECR_REPO:$BUILD_NUMBER .'
      }
    }
  }
}

```

BRANCH STRATEGY IN APNAKART

- push to develop → triggers dev environment deploy
- push to main → triggers uat pipeline with manual approval gate before prod

Q14 Explain the deployment strategies you have used in your project.

ROLLING UPDATE (DEFAULT IN APNAKART)

- Kubernetes gradually replaces old pods with new ones, defined in Deployment spec
- maxSurge: 1, maxUnavailable: 0 — ensures zero downtime during rollout
- Automatic rollback via kubectl rollout undo if health checks fail

BLUE/GREEN DEPLOYMENT

- Maintained two identical EKS deployments (blue = live, green = new version)
- Traffic switched by updating the Kubernetes Service selector label
- Instant rollback: revert the selector — no pod restart needed

CANARY DEPLOYMENT

- Deployed new version to 10% of pods initially; monitored error rate + latency in Datadog
- Gradually increased traffic split (10% → 30% → 100%) over defined time windows
- Aborted and rolled back automatically if error rate exceeded SLO threshold

WHEN TO USE WHICH

- Rolling: Standard releases with no hard cutover requirement
- Blue/Green: High-risk releases needing instant rollback capability
- Canary: Feature flags, A/B testing, or risk-mitigation for critical services

Q15 What do you do when you receive a P1 incident?

IMMEDIATE RESPONSE (0–5 MINUTES)

- Acknowledge the alert in PagerDuty/Slack immediately — SLA clock starts
- Declare incident in #incidents Slack channel; assign Incident Commander and Scribe roles
- Open a war-room bridge (Zoom/Meet) for real-time coordination

TRIAGE (5–15 MINUTES)

- Assess blast radius: How many users/services affected? Is data at risk?
- Check Datadog/Grafana for anomalies in the time window of incident onset
- Review recent deployments or config changes in last 30 minutes (git log, Jenkins history)

MITIGATION (BEFORE ROOT CAUSE)

- If recent deploy is suspect: kubectl rollout undo immediately — restore service first
- If infra issue: failover to standby region or secondary AZ
- Scale up resources if it's a load spike: kubectl scale or increase ASG desired count

POST-INCIDENT

- Write a Post-Mortem (blameless) within 24–48 hours: timeline, root cause, contributing factors
- Define action items with owners and deadlines to prevent recurrence
- Update runbooks and alerting thresholds based on learnings

Q16 How do you prevent two associates from running Terraform at the same time?

SOLUTION — DYNAMODB STATE LOCKING

- Terraform's S3 backend supports locking via a DynamoDB table
- When terraform apply runs, it writes a lock entry to DynamoDB
- Any concurrent terraform apply by another engineer receives: 'Error acquiring the state lock'

```
# backend.tf
terraform {
  backend "s3" {
    bucket = "my-tf-state-bucket"
    key = "apnakart/prod/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-state-lock" # LockID as partition key
    encrypt = true
  }
}
```

ADDITIONAL PROCESS CONTROLS

- All Terraform runs happen only through Jenkins/GitHub Actions pipelines — not local machines in prod
- Pipeline enforces serialization: only one pipeline job runs per workspace at a time
- Engineers submit PRs; pipeline runs terraform plan on PR open, terraform apply only after merge + approval
- If lock is stuck (e.g. engineer's laptop crashed): terraform force-unlock <LOCK_ID> — requires team lead approval

Q17 How do you set up a disaster recovery process for a zero-downtime application?

DR STRATEGY — ACTIVE/PASSIVE MULTI-REGION

- **Primary Region:** ap-south-1 (Mumbai) — active traffic
- **DR Region:** ap-southeast-1 (Singapore) — warm standby

DATA LAYER

- RDS Multi-AZ for local HA; cross-region read replica for DR promotion
- S3 Cross-Region Replication (CRR) for all data buckets
- RPO target: < 1 hour | RTO target: < 15 minutes

COMPUTE LAYER

- EKS cluster in DR region kept warm with minimal node count (scale-up on failover)
- Terraform modules are region-agnostic — identical infra deployable in any region
- Container images in ECR replicated to DR region repository

TRAFFIC LAYER

- Route53 health checks on primary ALB — automatic failover to DR ALB if health check fails
- CloudFront with origin failover group: primary S3/ALB + secondary DR origin

TESTING

- Quarterly DR drills: simulate AZ failure, promote DR region, validate < 15-min RTO
- Chaos engineering experiments (pod deletion, node drain) in staging to validate resilience
- Runbook documented and version-controlled; all steps are scripted, not manual